INFRAESTRUTURA DE HARDWARE

CPU

Torneio de Processadores

Simule o Ciclo Fetch-Decode-Execute em C ou Python

Atividade prática em sala

1. O que fazer?

Cada equipe vai construir um **simulador da MiniCPU** usando **C ou Python** (livre escolha do grupo). O simulador é um programa que imita o funcionamento interno de um processador real, executando o ciclo clássico:

FETCH → DECODE → EXECUTE

O programa de teste (os dados iniciais e as instruções em formato numérico) já estará **hardcoded** no próprio código do simulador. Não é necessário escrever assembly o foco é entender e implementar **como o processador busca, decodifica e executa** cada instrução.

Ao final, cada equipe apresenta brevemente o que fez (1–5 minutos), mostrando o trace de execução rodando.

2. Dinâmica

A atividade acontece inteiramente em sala, distribuída nas fases abaixo:

Fase	Duração	O que acontece
1 – Briefing	15 min	O professor apresenta a arquitetura MiniCPU, explica o ciclo F-D-E com um exemplo, distribui os desafios.
2 – Pesquisa e Planejamento	25 min	Equipes pesquisam sobre ciclo de instrução, definem a linguagem (C ou Python), esboçam a estrutura do código e distribuem tarefas entre os membros.
3 – Implementação	60 min	Mão na massa! Implementar o simulador com o ciclo Fetch-Decode-Execute completo. O programa de teste (dados iniciais na memória) já deve estar hardcoded no código para resolver o desafio do grupo.
4 – Teste e Trace	15 min	Testar o simulador, validar o resultado esperado e garantir que o trace de execução (estado dos registradores e memória a cada ciclo) está sendo exibido corretamente.
5 – Apresentações	20 min	Cada equipe apresenta rapidamente (1–2 min): qual o desafio, como estruturaram o F-D-E no código, e demonstração do trace rodando. O professor faz perguntas pontuais.
	2h	**Tempo total da atividade**

Importante: a Fase 2 (pesquisa) é intencional. Os alunos devem pesquisar sobre o ciclo de instrução, entender o conceito antes de programar. Isso reforça a autonomia e o aprendizado.

3. Arquitetura da MiniCPU

3.1 Registradores

- **R0, R1, R2, R3** – 4 registradores de propósito geral (inteiros 0–255)
- **PC** – Program Counter (endereço da próxima instrução, inicia em 0)
- **ZF** – Zero Flag (1 se última compare (CMP) resultou zero, 0 caso contrário)

3.2 Memória

256 posições (0x00 a 0xFF), cada uma armazenando um valor de 0 a 255. A memória é compartilhada: as instruções ficam a partir de 0x00 e os dados ficam tipicamente a partir de 0x08 ou 0x10. No código do simulador, basta usar um **array de 256 inteiros**.

3.3 Formato da Instrução

Cada instrução ocupa **3 posições** consecutivas na memória:

```
[opcode]  [operando1]  [operando2]
```

Instruções que usam menos operandos preenchem as posições restantes com 0. Por exemplo, HALT ocupa: 0x0A, 0x00, 0x00.

3.4 Conjunto de Instruções (ISA)

Opcode	Mnemônico	Operação	Descrição
0x01	LOAD R, addr	R ← MEM[addr]	Carrega valor da memória para registrador
0x02	STORE R, addr	MEM[addr] ← R	Armazena valor do registrador na memória
0x03	ADD R1, R2	R1 ← R1 + R2	Soma dois registradores
0x04	SUB R1, R2	R1 ← R1 - R2	Subtrai R2 de R1
0x05	MOV R, imm	R ← imm	Move valor imediato para registrador
0x06	CMP R1, R2	flags ← R1 - R2	Compara dois registradores (seta flags)
0x07	JMP addr	PC ← addr	Salto incondicional
0x08	JZ addr	if ZF: PC ← addr	Salta se flag zero ativa
0x09	JNZ addr	if !ZF: PC ← addr	Salta se flag zero inativa
0x0A	HALT	Encerra execução	Para o processador

Convenções: nos opcodes LOAD e STORE, operando1 é o índice do registrador (0=R0, 1=R1, 2=R2, 3=R3) e operando2 é o endereço de memória. Em ADD, SUB e CMP, ambos os operandos são índices de registradores. Em MOV, operando1 é o registrador e operando2 é o valor imediato. Em JMP, JZ e JNZ, operando1 é o endereço de destino.

4. O ciclo Fetch-Decode-Execute

O simulador deve executar um loop que repete estas três etapas até encontrar HALT:

FETCH (Busca)

O processador lê 3 posições consecutivas da memória a partir do PC:

```
opcode    = memoria[PC]
operando1 = memoria[PC + 1]
operando2 = memoria[PC + 2]
PC = PC + 3
```

Isso simula o processador buscando a instrução na memória e avançando o ponteiro.

DECODE (Decodificação)

O processador identifica **qual operação** o opcode representa e interpreta os operandos. No código, isso equivale a um **switch/case** (C) ou **if/elif** (Python) que verifica o valor do opcode.

EXECUTE (Execução)

O processador realiza a ação: acessa registradores, altera memória, atualiza flags ou modifica o PC (saltos). No código, cada case/elif executa a lógica correspondente.

Trace de execução: a cada ciclo, o simulador deve imprimir o estado atual qual instrução foi executada, valores dos registradores (R0–R3), PC, ZF, e opcionalmente as posições relevantes da memória. Exemplo de saída:

```
Ciclo 1: MOV R0, 10   | R0=10  R1=0   R2=0   R3=0   | PC=3  ZF=0
Ciclo 2: MOV R1, 1    | R0=10  R1=1   R2=0   R3=0   | PC=6  ZF=0
Ciclo 3: ADD R0, R1   | R0=11  R1=1   R2=0   R3=0   | PC=9  ZF=0
Ciclo 4: HALT         | R0=11  R1=1   R2=0   R3=0   | PC=12 ZF=0
```

5. Desafios por equipe

Cada equipe recebe um desafio computacional único. O grupo deve **pré-carregar na memória** do simulador tanto as instruções (em formato numérico/hexadecimal) quanto os dados iniciais, e então rodar o ciclo F-D-E até o HALT. O foco é demonstrar que o simulador funciona corretamente para o problema dado.

Equipe	Desafio	Descrição e Validação
Grupo 1	**Fibonacci**	Pré-carregar N=7 na memória (0x10). O simulador deve calcular o N-ésimo Fibonacci e gravar o resultado em 0x20. Validar com: fib(7) = 13.
Grupo 2	**Somatório de array**	Pré-carregar 8 valores nos endereços 0x10–0x17 (ex: 3, 7, 2, 5, 1, 8, 4, 6). O simulador deve somar todos e gravar o resultado em 0x20. Validar com: soma = 36.
Grupo 3	**Fatorial**	Pré-carregar N=5 em 0x10. Calcular N! por multiplicações via soma repetida e gravar em 0x20. Validar com: 5! = 120.
Grupo 4	**Máximo de array**	Pré-carregar 8 valores em 0x10–0x17 (ex: 12, 45, 7, 89, 23, 56, 3, 67). Encontrar o maior e gravar em 0x20. Validar com: max = 89.
Grupo 5	**Mínimo de array**	Pré-carregar 8 valores em 0x10–0x17 (ex: 34, 12, 56, 3, 78, 45, 9, 67). Encontrar o menor e gravar em 0x20. Validar com: min = 3.
Grupo 6	**Contagem condicional**	Pré-carregar limiar=20 em 0x08 e 8 valores em 0x10–0x17 (ex: 10, 25, 5, 30, 15, 40, 8, 22). Contar quantos são maiores que o limiar. Resultado em 0x20. Validar com: 4.
Grupo 7	**Média inteira**	Pré-carregar 8 valores em 0x10–0x17 (ex: 10, 20, 30, 40, 10, 20, 30, 40). Calcular a média (divisão por subtrações sucessivas de 8). Resultado em 0x20. Validar com: 25.
Grupo 8	**Potência**	Pré-carregar BASE=3 em 0x10 e EXP=4 em 0x11. Calcular BASE^EXP por multiplicações via soma repetida. Resultado em 0x20. Validar com: 81.
Grupo 9	**Cifra de césar**	Pré-carregar chave=3 em 0x08 e 8 bytes em 0x10–0x17 (ex: 65, 66, 67, ...). Somar a chave a cada byte e gravar em 0x20–0x27. Validar com: 68, 69, 70, ...
Grupo 10	**Contagem de pares**	Pré-carregar 8 valores em 0x10–0x17 (ex: 4, 7, 2, 9, 6, 1, 8, 3). Contar quantos são pares (subtrações de 2 até 0 ou 1). Resultado em 0x20. Validar com: 4.
Grupo 11	**Busca linear**	Pré-carregar alvo=56 em 0x08 e 8 valores em 0x10–0x17 (ex: 12, 34, 56, 78, 90, 11, 22, 33). Retornar índice (0–7) em 0x20 ou 0xFF se não encontrado. Validar com: 2.
Grupo 12	**Bubble Sort**	Pré-carregar 4 valores em 0x10–0x13 (ex: 40, 10, 30, 20). Ordenar in-place em ordem crescente. Validar com: 10, 20, 30, 40.

5.1 Como montar o programa na memória

O grupo deve traduzir manualmente a lógica do desafio para uma sequência de opcodes e operandos, e carregar tudo no array de memória no início do programa. Exemplo para "MOV R0, 10; MOV R1, 1; ADD R0, R1; HALT":

Python:

```
mem = [0] * 256
mem[0] = 0x05; mem[1] = 0x00; mem[2] = 0x0A   # MOV R0, 10
mem[3] = 0x05; mem[4] = 0x01; mem[5] = 0x01   # MOV R1, 1
mem[6] = 0x03; mem[7] = 0x00; mem[8] = 0x01   # ADD R0, R1
mem[9] = 0x0A; mem[10] = 0x00; mem[11] = 0x00 # HALT
```

C:

```
mem[0x00] = 0x05; mem[0x01] = 0x00; mem[0x02] = 0x0A;  /* MOV R0, 10 */
mem[0x03] = 0x05; mem[0x04] = 0x01; mem[0x05] = 0x01;  /* MOV R1, 1  */
mem[0x06] = 0x03; mem[0x07] = 0x00; mem[0x08] = 0x01;  /* ADD R0, R1 */
mem[0x09] = 0x0A; mem[0x0A] = 0x00; mem[0x0B] = 0x00;  /* HALT       */
```

Os dados do desafio (arrays, limiares, etc.) são carregados nas posições indicadas (0x08, 0x10–0x17, etc.) da mesma forma.

6. Esqueletos de código

Abaixo, esqueletos mínimos para iniciar. **O grupo deve completar todas as instruções e adicionar o trace de execução.**

6.1 Esqueleto python clique aqui

```
class MiniCPU:
    def __init__(self):
        self.mem = [0] * 256
        self.reg = [0, 0, 0, 0]  # R0-R3
        self.pc = 0
        self.zf = 0
        self.running = True
        self.ciclo = 0

    def fetch(self):
        op = self.mem[self.pc]
        a  = self.mem[self.pc + 1]
        b  = self.mem[self.pc + 2]
        self.pc += 3
        return op, a, b

    def decode_execute(self, op, a, b):
        if   op == 0x01: self.reg[a] = self.mem[b]          # LOAD
        elif op == 0x02: self.mem[b] = self.reg[a]          # STORE
        elif op == 0x03:                                    # ADD
            self.reg[a] = (self.reg[a] + self.reg[b]) & 0xFF
        elif op == 0x04:                                    # SUB
            self.reg[a] = (self.reg[a] - self.reg[b]) & 0xFF
        elif op == 0x05: self.reg[a] = b                    # MOV
        elif op == 0x06:                                    # CMP
            self.zf = 1 if self.reg[a] == self.reg[b] else 0
        elif op == 0x07: self.pc = a                        # JMP
        elif op == 0x08:                                    # JZ
            if self.zf: self.pc = a
        elif op == 0x09:                                    # JNZ
            if not self.zf: self.pc = a
        elif op == 0x0A: self.running = False               # HALT

    def trace(self, op, a, b):
        nomes = {1:'LOAD',2:'STORE',3:'ADD',4:'SUB',
                 5:'MOV',6:'CMP',7:'JMP',8:'JZ',9:'JNZ',10:'HALT'}
        nome = nomes.get(op, '???')
        print(f'Ciclo {self.ciclo}: {nome:5s} {a},{b} |'
              f' R0={self.reg[0]:3d} R1={self.reg[1]:3d}'
              f' R2={self.reg[2]:3d} R3={self.reg[3]:3d}'
              f' | PC={self.pc:3d} ZF={self.zf}')
```

```
    def run(self):
        while self.running and self.pc < 256:
            self.ciclo += 1
            op, a, b = self.fetch()
            self.decode_execute(op, a, b)
            self.trace(op, a, b)
```

6.2 Esqueleto C clique aqui

```
#include <stdio.h>
#include <stdint.h>

uint8_t mem[256] = {0};
uint8_t reg[4] = {0};
uint8_t pc = 0, zf = 0, running = 1;
int ciclo = 0;

void fetch(uint8_t *op, uint8_t *a, uint8_t *b) {
    *op = mem[pc]; *a = mem[pc+1]; *b = mem[pc+2];
    pc += 3;
}

void decode_execute(uint8_t op, uint8_t a, uint8_t b) {
    switch (op) {
        case 0x01: reg[a] = mem[b]; break;
        case 0x02: mem[b] = reg[a]; break;
        case 0x03: reg[a] = reg[a] + reg[b]; break;
        case 0x04: reg[a] = reg[a] - reg[b]; break;
        case 0x05: reg[a] = b; break;
        case 0x06: zf = (reg[a] == reg[b]) ? 1 : 0; break;
        case 0x07: pc = a; break;
        case 0x08: if (zf) pc = a; break;
        case 0x09: if (!zf) pc = a; break;
        case 0x0A: running = 0; break;
    }
}

void trace(uint8_t op, uint8_t a, uint8_t b) {
    const char *nomes[] = {"","LOAD","STORE","ADD",
        "SUB","MOV","CMP","JMP","JZ","JNZ","HALT"};
    printf("Ciclo %d: %-5s %d,%d | R0=%3d R1=%3d"
           " R2=%3d R3=%3d | PC=%3d ZF=%d\n",
           ciclo, nomes[op], a, b,
           reg[0], reg[1], reg[2], reg[3], pc, zf);
}
```

```
int main() {
    // TODO: carregar programa e dados na mem[]
    while (running && pc < 256) {
        uint8_t op, a, b;
        ciclo++;
        fetch(&op, &a, &b);
        decode_execute(op, a, b);
        trace(op, a, b);
    }
    return 0;
}
```

7. Avaliação

A pontuação desta atividade é **a critério do professor**, baseada na **participação** observada ao longo da aula. Os aspectos que serão observados incluem:

- **Engajamento da equipe:** todos os membros estão contribuindo ativamente? Há divisão de tarefas?
- **Compreensão do F-D-E:** a equipe demonstra entender a separação entre Fetch (buscar da memória), Decode (interpretar o opcode) e Execute (realizar a operação)?
- **Progresso técnico:** o simulador roda? O trace mostra a execução coerente? O resultado está correto?
- **Apresentação:** a equipe consegue explicar o que fez em 1–5 minutos?
- **Atitude investigativa:** a equipe pesquisou, fez perguntas, tentou resolver problemas antes de pedir ajuda?

Uma equipe que implementou o ciclo F-D-E corretamente e demonstra compreensão, mesmo sem resolver o desafio completo, pode ter uma boa avaliação. O objetivo é **aprender fazendo**.

8. Regras e orientações

1. **Linguagem:** Python ou C, livre escolha do grupo.
2. **Equipes:** 5 membros. Todos devem participar da implementação e da apresentação.
3. **Pesquisa permitida:** podem usar internet, livros, material de aula. Não é permitido usar código de simulador pronto copiado integralmente.
4. **Sem bibliotecas de emulação:** o simulador deve ser construído do zero com lógica básica (arrays, loops, condicionais).
5. **Trace obrigatório:** o simulador deve imprimir o estado a cada ciclo F-D-E.
6. **Apresentação:** 1–5 minutos por equipe. Mostrem o código rodando e expliquem a estrutura.

9. Dicas para as equipes

- **Começem simples:** implementem primeiro só MOV + HALT. Quando funcionar, adicionem ADD, SUB, etc.
- **Dividam tarefas:** 2 pessoas na estrutura do simulador (fetch, loop principal), 2 nas instruções (decode/execute), 1 no trace e teste.
- **O trace é seu debugger:** se algo der errado, o trace mostra exatamente onde. Imprimam tudo!
- **Não existe divisão no ISA.** Para dividir, usem subtrações repetidas (ex: 25/8 = subtrair 8 de 25 até ficar < 8, contando quantas vezes).
- **Testem com o exemplo da Seção 5.1** antes de atacar o desafio do grupo.